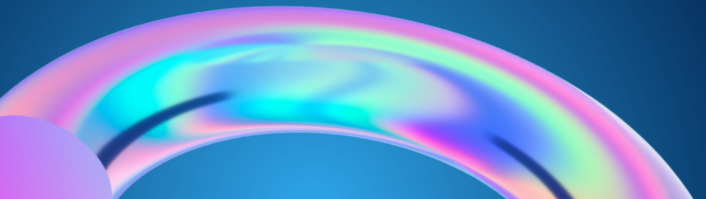
Summarizer

2023-03-07 Valter

Java 8 Interfaces

To bring functional programming to the Java language, several changes within the Collections API were made

.

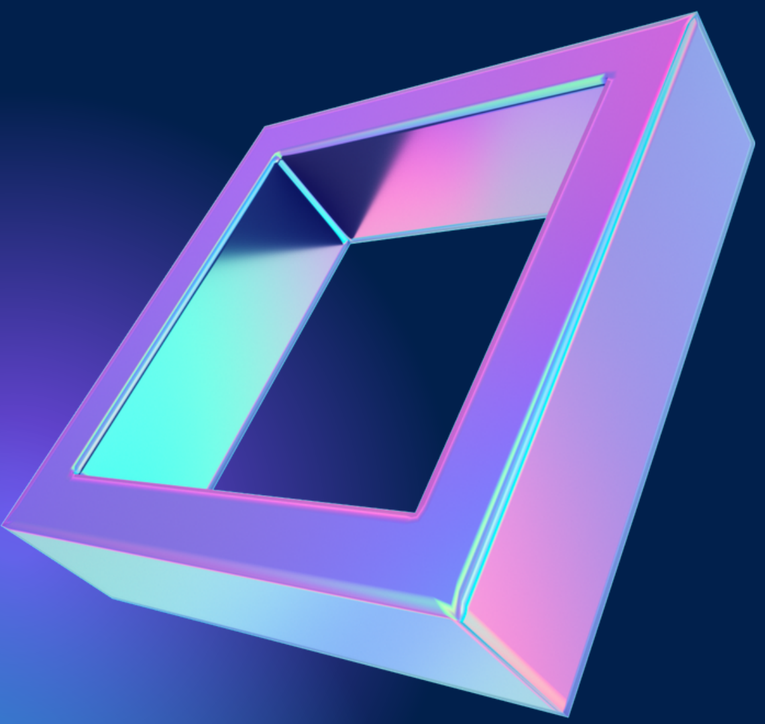


problems...solutions

Default methods

The diamond problem

The three rules

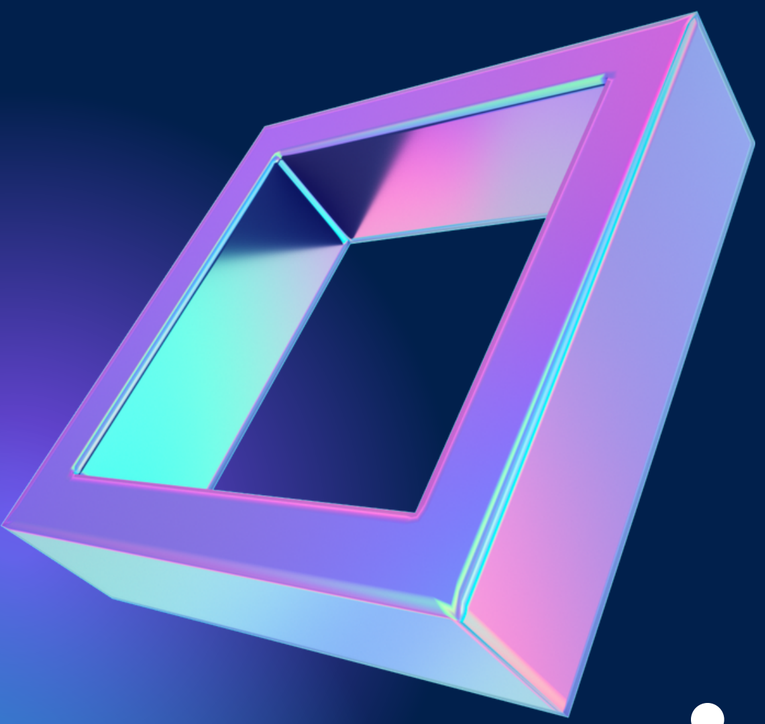


Default methods

General-purpose implementations
In most cases there's no need for overriding.

The three rules

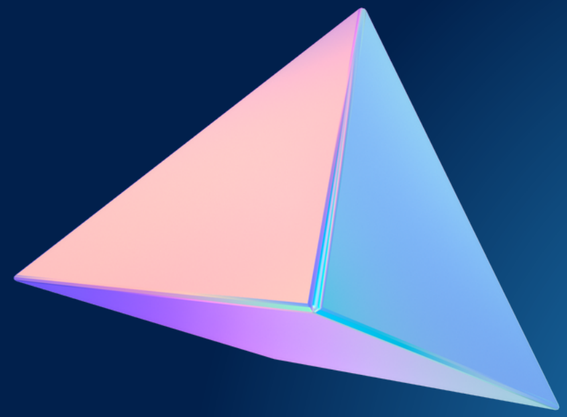
- 1. Never talk about fight Club
- 2. Fallow rule 1
- 3. There is no *No rule 3*



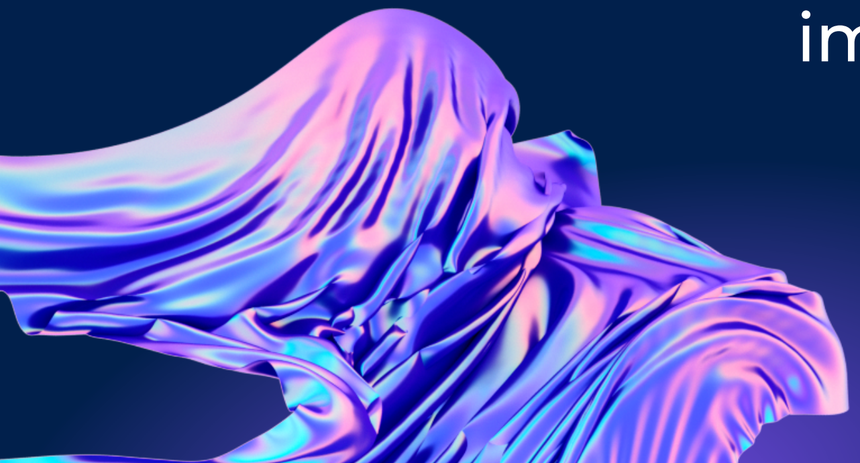
The three rules

- 1. Classes always win
- 2. Subtype wins over supertype
- 3. *No rule 3*

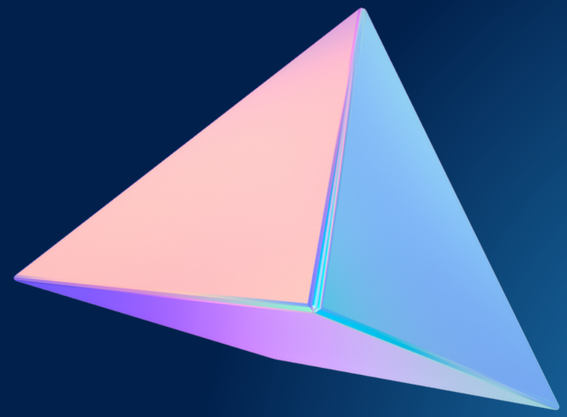
Static Methods



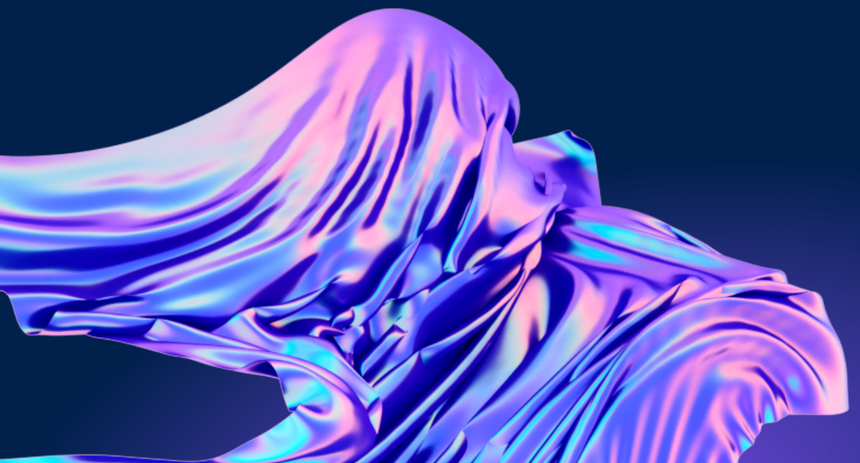
Another way of providing implementation code to interfaces is static methods.



The way we iterate until now is known as external iteration.



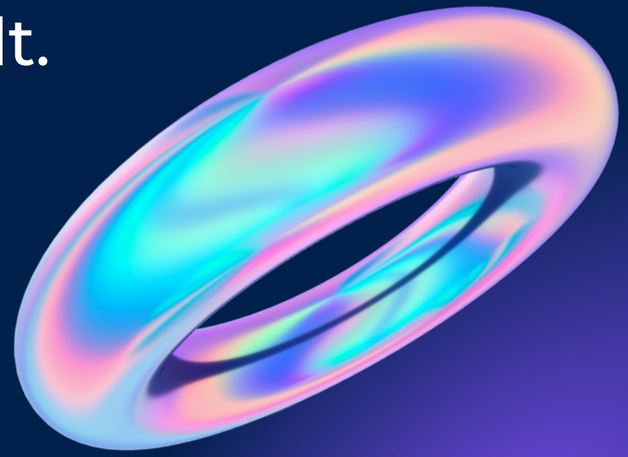
For loops / Enhanced Loops

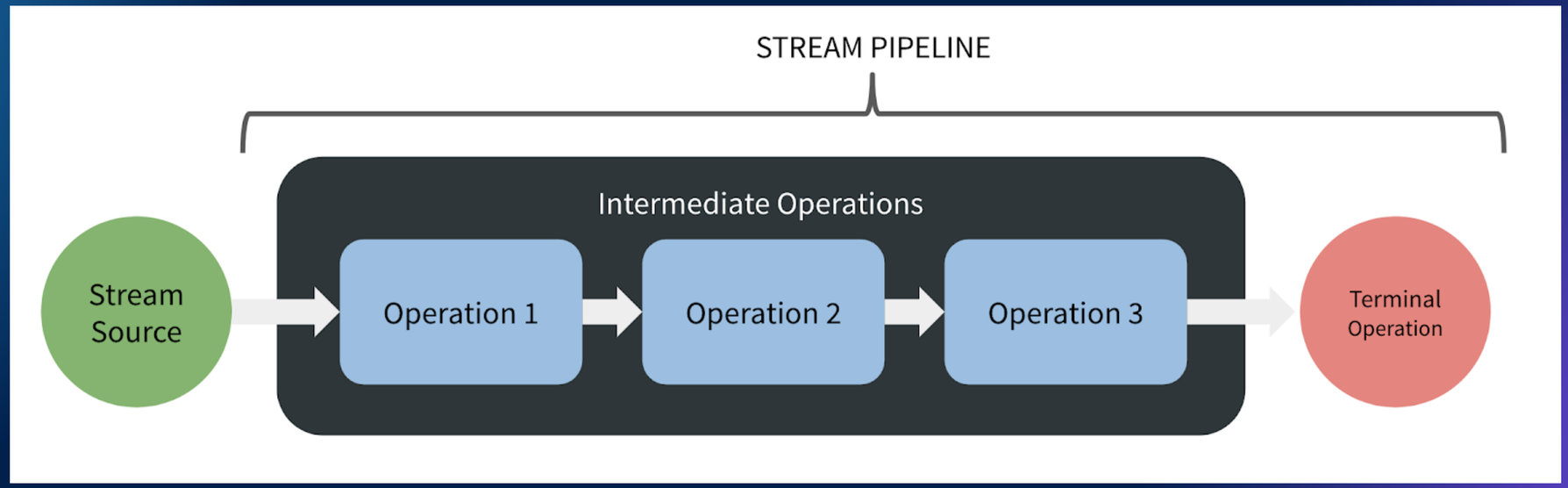



THE BETTER WAY
JUST AHEAD

Streams

A Stream is an abstraction for a sequence of elements from a source, supporting aggregate operations leading to a desired result.




STREAM PIPELINE
Intermediate Operations
Stream Source
Operation 1
Operation 2
Operation 3
Terminal Operation

```java
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class Main {
    public static void main(String[] args) {

        String message = "I'll send an SOS to the garbage world, " +
                "I hope that someone garbage gets my message in a garbage bottle.";

        String newMessage = Stream.of(message.split(" "))
                .filter(word -> !word.equals("garbage"))
                .map( word -> word.replace("message", "garbage").toUpperCase() )
                /* with Collector  */   .collect(Collectors.joining(" "));

        /* with reduce  * .reduce("", (s, s2) -> s + s2 + " ");*/
        System.out.println(newMessage);


    }
}
```

Building numeric streams

Java 8 primitive streams provide some useful static methods to create a numeric stream.

`range`(exclusive) and `rangeClosed`(inclusive)

```
IntStream evenNumbers = IntStream.range(0, 100)
                                 .filter(number -> number % 2 == 0);
```

So far we've seen streams created from a collection of elements or, just now, from a range of integers.

Infinite Streams!

Stream.iterate and Stream.generate are two methods allowing to create a stream from a function.

Since stream elements are produced **on demand** these two operations can produce elements *forever*.

`terate()` takes an initial value and a `java.util.function.UnaryOperator` to apply on each new value produced.

```
Stream<Integer> tens = Stream.iterate(0, n -> n + 10);
```

`generate()` takes a `java.util.function.Supplier` which will be called on demand.

```
IntStream randoms = IntStream.generate(() -> (int)(Math.random() * 100))
                             .limit(5); // limit bounds the number of elements
                                        // produced by the stream
```


Let the game begin.


THANK YOU!
www.allfreethankyounotes.com